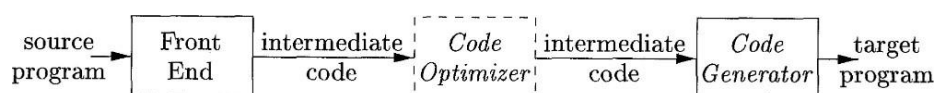


Code Generator

- **Input:** IR + symbol table; **Output:** target program
- There is often an optimization phase before code generation
- **Three primary tasks of a code generator:**
 - Instruction selection
 - Register allocation and assignment
 - Instruction ordering



- 代码生成器以编译器前端产生的中间表示（IR）以及相关符号表信息作为输入，并产生语义等价的目标程序作为输出，如图 8.1 所示。
- 对代码生成器的要求非常严格。目标程序必须保留源程序的语义并且是高质量的；也就是说，它必须有效地利用目标机器的可用资源。此外，代码生成器本身必须高效运行。
- 挑战在于，从数学上来说，为给定源程序生成最优目标程序的问题是不可判定的；代码生成中遇到的许多子问题（例如寄存器分配）都具有难以处理的计算复杂性。在实践中，我们必使用那些能够产生良好但不一定是最佳代码的启发式技术。幸运的是，启发式方法已经足够成熟，精心设计的代码生成器可以生成比普通代码生成器快几倍的代码。
- 要生成高效目标程序的编译器在代码生成之前包括一个优化阶段。优化器将 IR 映射到另一个可用于产生高效代码的 IR。一般来说，编译器的代码优化和代码生成步骤（通常称为后端）它们可能会在生成目标程序之前对 IR 进行多次处理。

第 9 章详细讨论了代码优化。无论代码生成之前有没有优化步骤，都可以使用本章介绍的技术。

- 代码生成器具有三个主要任务：指令选择、寄存器分配和指派、以及指令排序。第 8.1 节概述了这些任务的重要性。指令选择涉及选择适当的目标机器指令来实现 IR 语句。寄存器分配和指派涉及决定在哪些寄存器中保留哪些值。指令排序涉及决定以什么顺序安排指令的执行。

Design Issues

- **Design goals:**
 - Correctness (the most important)
 - Ease of implementation, testing, and maintenance
- **Many choices for the input IR:**
 - **Three-address representations:** quadruples, triples, indirect triples
 - **VM representations:** bytecodes and stack-machine code
 - **Graphical representations:** syntax trees and DAG's
- **The target program:**
 - RISC (reduced instruction set computer), CISC (complex instruction...)
 - Absolute machine-language programs; relocatable machine-language programs; assembly-language programs

虽然代码生成器的涉及依赖于中间表示形式、目标语言和运行时系统的特定细节，但指令选择、寄存器分配和指派以及指令排序等任务会在几乎所有的代码生成器设计中碰到。

- ◆ 代码生成器最重要的标准是生成正确的代码。
 - 正确性问题非常突出的原因是代码生成器可能会碰到很多种特殊情况。
 - 在优先考虑正确性的情况下，另一个重要的设计目标是把代码生成器设计得易于实现、测试和维护。

代码生成器的输入是前端生成的源程序的中间表示，以及符号表中的信息。这些信息用于确定 IR 中的名称所指的数据对象的运行时刻地址。

◆ IR 的表示形式有多种选择，包括：

- 三地址表示，例如四元组、三元组、间接三元组；
- 虚拟机表示，例如字节码和堆栈机代码；
- 线性表示，例如后缀表示法；
- 语法树和 DAG 等图形表示。本章中的许多算法都是根据第 6 章中考虑的表示形式来表达的：三地址码、树和 DAG。然而，我们讨论的技术也可以应用于其他中间表示。

在本章中，我们假设前端已经将源程序扫描、解析并翻译成一个相对较低级别的 IR，使得 IR 中出现的名称的值可以用目标机直接处理的量来表示。这些量可以是整数和浮点数等。我们还假设所有语法和静态语义错误都已经被检测出来，必要的类型检查都已经完成，并且类型转换运算已经被插入到必要的地方。因此，代码生成器可以在工作过程中假设它的输入已经排除了这些错误。

◆ 目标程序所依托的体系架构和具体的语言形式

- 目标机器的指令集体系结构对构建产生高质量机器代码的良好代码生成器的难度有重大影响。最常见的目标机架构是 RISC（精简指令集计算机）、CISC（复杂指令集计算机）和基于堆栈的。
- RISC 机器通常具有许多寄存器、三地址指令、简单的寻址模式和相对简单的指令集架构。
- 相比之下，CISC 机器通常具有很少的寄存器、双地址指令、多种寻址模式、多个寄存器类别、可变长度指令以及具有副作用的指令。
- 在基于堆栈的机器中，运算是通过将运算分量压入堆栈，然后对堆栈顶部的运算分量进行运算来完成的。为了实现高性能，栈部元素通常保存在寄存器中。因为人们认为堆栈组织过于限制并且需要大量交换和复制操作，所以基于堆栈的机器几乎消失了。

然而，随着 Java 虚拟机 (JVM) 的引入，基于堆栈的体系结构又复活了。JVM 是 Java 字节码的软件解释器，Java 字节码是 Java 编译器生成的中间语言。这个解释器提供跨多个平台的软件兼容性，这是 Java 成功的一个主要因素。为了克服解释的高性能损失

(大约是 10 倍的数量级)，即时 (JIT) Java 编译器应运而生。这些 JIT 编译器在运行时将字节码转换为目标机器的本机硬件指令集。提高 Java 性能的另一种方法是构建一个编译器，把 Java 程序直接编译成目标机器的机器指令彻底绕过 Java 字节码。

- 目标程序具体的形式：
- 生成一个使用绝对地址的机器语言程序作为输出的优点是程序可以放置在内存中某个固定的位置上，并立即执行。程序可以快速编译和执行。
- 输出可重定位的机器语言程序（通常称为目标模块）可以使各个子程序能够被分别编译。一组可重定位的目标模块可以被一个链接加载器链接在一起并加载运行。虽然我们要生成可重定位的目标模块，必须为链接和加载付出代价。但这样做可以使我们获得很多的灵活性。我们可以把子程序分开编译，并从一个目标模块调用其他先前已编译好的程序。如果目标机没有自动处理重定位，编译器就必须向加载器提供明确的重定位信息，以便把分开编译的程序模块链接起来。

生成汇编语言程序作为输出使得代码生成过程更加容易。我们可以生成符号指令并使用汇编器的宏功能来帮助生成代码。这么做的代价是代码生成后还需要增加一个汇编步骤。

在本章中，我们将使用一个非常简单的类 RISC 计算机作为目标机器。我们添加了一些类似 CISC 的寻址模式，以便我们还可以讨论 CISC 机器的代码生成技术。为了可读性，我们使用汇编代码作为目标语言。只要可以根据符号表中存储的偏移量和存放于符号表中的其他信息计算出来，代码生成器就可以为源程序中的名字生成可重定位地址或绝对地址。这和生成符号地址一样，都是很简单的东西。

A Simple Target Machine Model

- A three-address machine with load and store, computation, jump, and conditional jump operations

Type	Form	Effect
Load	LD <i>dst</i> , <i>addr</i>	load the value in location <i>addr</i> into location <i>dst</i> , where <i>dst</i> is often a register
Store	ST <i>x</i> , <i>r</i>	store the value in register <i>r</i> into the location <i>x</i>
Computation	OP <i>dst</i> , <i>src</i> ₁ , <i>src</i> ₂	apply the operation <i>OP</i> to the values in locations <i>src</i> ₁ and <i>src</i> ₂ , and place the result in location <i>dst</i>
Unconditional jumps	BR <i>L</i>	jump to the machine instruction with label <i>L</i>
Conditional jumps	Bcond <i>r</i> , <i>L</i>	jump to label <i>L</i> if the value in register <i>r</i> pass the test <i>Bcond</i> , e.g., less than zero

熟悉目标机器及其指令集是设计良好的代码生成器的先决条件。不幸的是，在代码生成的一般讨论中，不可能足够详细地描述任何目标机器来为该机器上的完整语言生成良好的代码。在本章中，我们将使用一个简单计算机的汇编代码作为目标语言。这个计算机是许多寄存器机器的代表。然而，本章介绍的很多代码生成技术也可以用于许多其他类别的机器。

我们的目标计算机是一个三地址机器的模型，具有加载和存储操作、计算操作、跳转操作和条件跳转。这个计算机的内存按照字节寻址，它具有 n 个通用寄存器 $R_0; R_1; \dots; R_n$ 。一个完整的汇编语言具有几十到上百个指令。为了避免因为过多的细节而妨碍对概念的理解，我们将使用一个非常有限的指令集，并假设所有的运算分量都是整数。大多数指令由一个运算符、一个目标地址、一个源运算分量的列表组成。指令之前可能有标号。我们假设有以下类型的指令可用：

- 加载操作：指令 LD *dst*,*addr* 将地址 *addr* 中的值加载到位置 *dst* 中。该指令表示赋值 $dst = addr$ 。该指令最常见的形式是 LD *r*,*x*，它将位置 *x* 中的值加载到寄存器 *r* 中。LD *r*₁,*r*₂ 形式的指令是寄存器到寄存器的复制，其中寄存器 *r*₂ 的内容被复制到寄存器 *r*₁ 中。

- 存储操作：指令 $ST\ x, r$ 将寄存器 r 中的值存储到位置 x 中。该指令表示赋值 $x = r$ 。

- 计算操作：形如 $OP\ dst, src1, src2$ ，其中 OP 是 ADD 或 SUB 等运算符，而 dst 、 $src1$ 和 $src2$ 是内存位置，这些位置不一定要相互不同。该机器指令的作用是将 OP 表示的运算应用于位置 $src1$ 和 $src2$ 中的值，并将该运算的结果放置在位置 dst 中。例如， $SUB\ r1; r2; r3$ 计算 $r1 = r2 - r3$ 。先前存储在 $r1$ 中的任何值丢失了，但如果 $r1$ 等于 $r2$ 或 $r3$ ，则计算机首先读取原来的值。只需要一个运算分量的单目运算符没有 $src2$ 。

- 无条件跳转：指令 $BR\ L$ 使控制转移到带有标号 L 的机器指令。（ BR 代表分支。）

- 条件跳转：形式为 $Bcond\ r, L$ ，其中 r 是寄存器， L 是一个标号，而 $cond$ 代表对寄存器 r 中的值所做的某个常见测试。例如，如果寄存器 r 中的值小于零， $BLTZ\ r, L$ 会导致控制跳转到标号 L ，如果不小于零，则允许控制传递到下一条机器指令。

Addressing Modes (寻址模式)

In instructions, a location can be:

- **Variable name x** : the memory location reserved for x (x 's l -value)
- **$a(r)$** : a is a variable and r is a register; the memory location is computed by taking the l -value of a and adding to it the value in register r (this is very useful for accessing arrays)

我们假设我们的目标机器有多种寻址模式：

- 在指令中，一个位置可以是变量名 x ，它指向分配给 x 的内存位置（即 x 的左值）。
- 位置也可以是一个带有下标的形如 $a(r)$ 的地址，其中 a 是变量， r 是寄存器。 $a(r)$ 表示的内存位置是通过获取 a 的左值加上存放在寄存器 r 中的值来计算的。例如，指令 $LD\ R1, a(R2)$ 的效果是 $R1 = \text{contents}(a + \text{contents}(R2))$ ，其中 $\text{contents}(x)$ 表示由 x 所表示的寄存器或内存位置中存放的内容。这个寻址模式对于访问数组很有用，其中 a 是数组的基地址（即第一个元素的地址）， r 保存从基地址到数组 a 的某个元素所要经过的字节数。

Addressing Modes (寻址模式)

In instructions, a location can be:

- *** r** : the memory location found in the location represented by the contents of register r
 - $LD\ R1, *R2$ has the effect: $R1 = \text{contents}(\text{contents}(\text{contents}(R2)))$
- ***constant(r)**: the memory location found in the location obtained by adding the constant to the contents of r (e.g., $*100(r)$)
- **#constant**: immediate constant addressing mode
 - $LD\ R1, \#100$ loads the integer 100 into register $R1$

我们还支持另两种间接寻址模式：

- $*r$ 表示在寄存器 r 的内容所表示的位置上存放的内存位置，
- $*100(r)$ 表示在寄存器 r 的内容加 100 的和所代表的位置上的内容所代表的位置。

例如， $LD\ R1, *100(R2)$ 的作用是设置 $R1 = \text{contents}(\text{contents}(100 + \text{contents}(R2)))$ ，即首先将寄存器 $R2$ 中的内容加上 100 的和，取出和值所指向位置中的内容，再把这个内容代表的位置中的值加载到 $R1$ 中。

- 最后，我们支持一个直接常数寻址模式。该常数以 # 开头。指令 LD R1, #100 将整数 100 加载到寄存器 R1 中，而 ADD R1, R1, #100 将整数 100 加到寄存器 R1 中。

指令之后的注释由 // 开头

Examples (1)

- $x = y - z$

```
LD R1, y          // R1 = y
LD R2, z          // R2 = z
SUB R1, R1, R2    // R1 = R1 - R2
ST x, R1          // x = R1
```

- $b = a[i]$

```
LD R1, i          // R1 = i
MUL R1, R1, 8     // R1 = R1 * 8
LD R2, a(R1)      // R2 = contents(a + contents(R1))
ST b, R2          // b = R2
```

- 三地址指令 $x=y-z$ 可用机器指令序列来实现：

也许我们可以做得更好。良好的代码生成算法的目标之一是尽可能避免使用所有这四个指令。例如， y 和/或 z 可能已被计算出来并存放在一个寄存器中，如果是这样，我们就可以避免相应的 LD 步骤。同样，如果 x 的值被使用时都存放在寄存器中，并且以后不再被用到，我们就不需要把这个值保存回 x 。

- 假设 a 是一个数组，其元素是 8（比如实数）字节值。还假设 a 的元素下标从 0 开始。我们可以通过下面的指令序列执行三地址指令 $b = a[i]$ ：

也就是说，第二步计算 $8i$ ，第三步将 a 的第 i 个元素中的值放入寄存器 R2 中，这个元素位于离数组 a 的基地址 $8i$ 字节的地方。

Examples (2)

- $a[j] = c$

```
LD  R1, c           // R1 = c
LD  R2, j           // R2 = j
MUL R2, R2, 8        // R2 = R2 * 8
ST  a(R2), R1        // contents(a + contents(R2)) = R1
```

- $x = *p$

```
LD  R1, p           // R1 = p
LD  R2, 0(R1)        // R2 = contents(0 + contents(R1))
ST  x, R2            // x = R2
```

- 类似地，对三地址指令 $a[j] = c$ 表示的对数组 a 的赋值可以实现为：
- 要实现简单的指针间接寻址，例如三地址语句 $x = *p$ ，我们可以使用如下机器指令：

Examples (3)

- $*p = y$

```
LD  R1, p           // R1 = p
LD  R2, y           // R2 = y
ST  0(R1), R2       // contents(0 + contents(R1)) = R2
```

- if $x < y$ goto L

```
LD  R1, x           // R1 = x
LD  R2, y           // R2 = y
SUB  R1, R1, R2      // R1 = R1 - R2
BLTZ R1, M          // if R1 < 0 jump to M
```

M is a label that represents the first machine instruction generated from the three-address instruction that has label L

- 通过指针的赋值语句 $*p = y$ 可以类似地用以下的机器代码实现：
- 最后，考虑一个条件跳转三地址指令，例如 if $x < y$ goto L

等效的机器代码类似于：

这里，M 是从标号为 L 的三地址指令所产生的机器指令序列中的第一个指令的标号。

对于任何三地址指令，我们希望可以省略这些指令中的某些指令，省略的原因是所需要的运算分量已经在寄存器中，也可能因为结果不需要存放回内存。

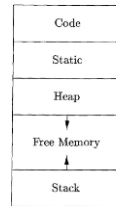
Addresses in the Target Code

- How to generate code for procedure calls and returns?
 - **Static allocation** (静态分配)
 - **Stack allocation** (栈式分配)
- How to replace names in IR by code to access storage locations?

在第 7.1 节中，我们描述了每个正在执行的程序是如何在其逻辑地址空间上运行，该空间被划分为四个代码及数据区域：

1. 静态确定的区域代码，保存可执行目标代码。目标代码的大小可以在编译时确定。
2. 静态确定的数据区 `Static`，用于保存全局常量和编译器生成的其他数据。全局常量和编译器数据的大小也可以在编译时确定。
3. 动态管理区域堆，用于保存程序执行期间分配和释放的数据对象。堆的大小无法在编译时确定。
4. 动态管理区域栈，用于保存在过程调用和返回期间创建和销毁的活动记录。与堆一样，栈的大小无法在编译时确定。

Static Allocation (静态分配)



- The size and layout of activation records are determined by the code generator via the information in the symbol table
 - Constant *staticArea* gives the address of the beginning of an activation record
- Target program code for the three-address code *call callee*

```
ST  callee.staticArea, #here + 20  
BR  callee.codeArea
```

Store the literal *return address* (the address of the instruction after BR) at the beginning of the callee's activation record

Constant *codeArea* gives the address of the first instruction of the callee in the Code area of the run-time memory

为了说明简化过程调用和返回的代码生成，我们将重点关注以下三地址语句：

call callee

return

halt

action，这是代表其他三地址语句的占位符

- 活动记录的大小和布局由代码生成器通过符号表中存储的名字信息来确定。

我们将首先说明如何在过程调用时在一个活动记录中存储返回地址，以及在过程调用结束后如何把控制返回到这个地址。为了方便起见，我们假设活动记录中的第一个位置保存返回地址。

- 让我们首先考虑实现最简单情况（静态分配）所需的代码。这里，中间代码中的 *call callee* 语句可以通过两个目标机器指令的序列来实现：

ST 指令把返回地址保存在被调用者活动记录的开始处，BR 将控制传递到被调用过程的目标代码上。属性 *callee.staticArea* 是一个常量，它给出了 *callee* 的活动记录的开始地址，属性 *callee.codeArea* 也是一个常量，指向运行时内存中的 code 区中被调用过程 *callee* 的第一条指令的地址。

ST 指令中的运算分量 $\#here + 20$ 是返回地址的文字表示；它是紧跟在 BR 指令之后的指令的地址。我们假设 $\#here$ 是当前指令的地址，并且调用序列中的三个常量

(*callee.staticArea*, *callee.codeArea*, $\#here + 20$) 加上两条指令 (ST, BR) 的长度为 5 个字，即 20 个字节。

Static Allocation (静态分配)

- Code for the *return* statement in a *callee*

BR **callee.staticArea*

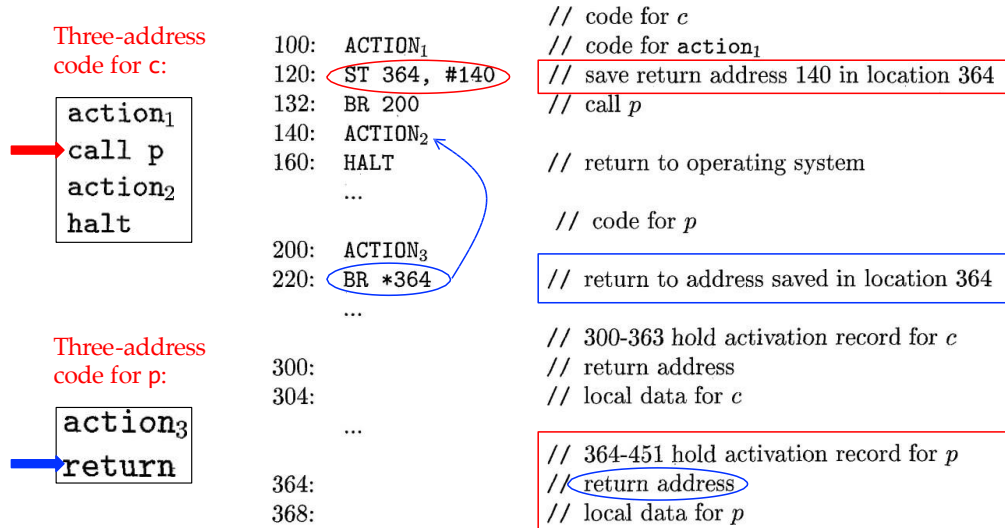


Transfer control to the address saved at the beginning of the activation record for *callee*

过程代码的结尾处是一个返回到调用过程的指令，但是没有调用者的第一个过程例外，它的最后一个指令是 HALT，它将控制权返回给操作系统。 *return* 语句可以通过简单的跳转指令 BR **callee.staticArea* 来实现，该指令将控制转移到被调用者活动记录开头位置的地址上。

Example

Note: Here the return address is stored at the beginning of the callee's activation record, which is different from Chapter 6. This is fine since the order among actual parameters, returned values, and saved machine status does not matter.



• 图 8.4 显示了该三地址代码的目标程序。我们使用伪指令 ACTION 来表示执行语句 action 的机器指令序列，它表示与本讨论无关的三地址代码。我们假定过程 *c* 的代码从地址 100 处开始，过程 *p* 的代码从地址 200 处开始。我们假设每个 ACTION 伪指令占用 20 个字节。我们进一步假设这些过程的活动记录以静态方式分配，其位置分别是 300 和 364。

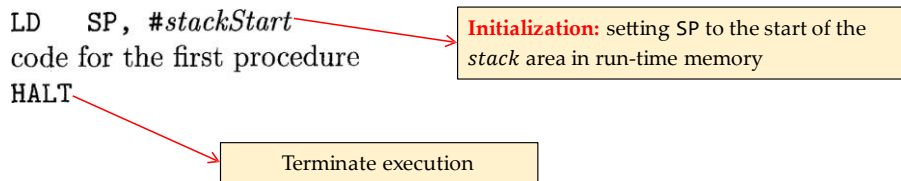
• 从地址 100 开始的指令实现了过程 *c* 的语句。

• 因此，程序的运行从地址 100 上的指令 ACTION₁ 开始。地址 120 处的 ST 指令将返回地址 140 保存在机器状态字段中，这是 *p* 的活动记录中的第一个字。地址 132 处的 BR 指令把控制转移到被调用过程 *p* 的目标代码中的第一条指令。

执行 ACTION₃ 后，位于地址 220 的跳转指令被执行。由于上述调用序列将位置 140 保存在地址 364 处，因此当执行地址 220 处的 BR 语句时，*364 代表 140。因此，当过程 *p* 终止时，控制返回到地址 140，过程 *c* 继续执行。

Stack Allocation (栈式分配)

- Static allocation uses **absolute addresses**
- Static allocation can become stack allocation by using **relative addresses** for storage in activation records
 - Maintain in **a register SP** a pointer to **the beginning of the activation record on top of the stack**
- The code for the first procedure (**main**)



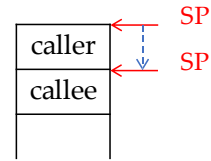
- 如果在保存活动记录时使用相对地址，静态分配就可以变栈分配。
- 然而，在栈分配中，只有等到运行时才能知道一个过程的活动记录的位置。

该位置通常存储在寄存器中，因此活动记录中的字可以通过相对于寄存器中值的偏移量来访问。目标机器的下标地址模式可以方便地完成这种访问。

- 活动记录的相对地址可以用相对于活动记录中的任一已知位置的偏移量来表示，正如我们在第 7 章中看到的那样。那么，我们将在寄存器 SP 中维护一个指向栈顶活动记录开始处的指针，这样就可以使所有的偏移量都是正偏移量。当过程调用发生时，调用过程增加 SP 的值，并将控制权转移给被调用过程。当控制权返回给调用者后，我们减少 SP 的值，从而释放被调用过程的活动记录。

- 第一个过程的代码通过将 SP 设置为内存中栈区的开始位置，完成对栈的初始化：

Stack Allocation (栈式分配)



- A procedure **calling sequence** Additional work comparing to static allocation

Each takes 4 bytes	<u>ADD</u> <u>SP</u> , <u>SP</u> , # <i>caller.recordSize</i>	<code>// increment stack pointer</code>
	<u>ST</u> <u>*SP</u> , # <i>here</i> + 16	<code>// save return address *</code>
	<u>BR</u> <u>callee.codeArea</u>	<code>// return to caller</code>

- The **return sequence**

<code>BR *0(SP)</code>	<code>// return to caller (done in callee)</code>
<code>SUB SP, SP, #<i>caller.recordSize</i></code>	<code>// decrement stack pointer</code> (done in caller)

Additional work comparing to static allocation

* Return address is at the beginning of the activation record

- 一个过程调用序列递增 SP 的值，保存返回地址，并将控制权转移到被调用的过程：

运算分量#*caller.recordSize* 表示一条活动记录的大小，因此 ADD 指令使 SP 指向下一条活动记录。

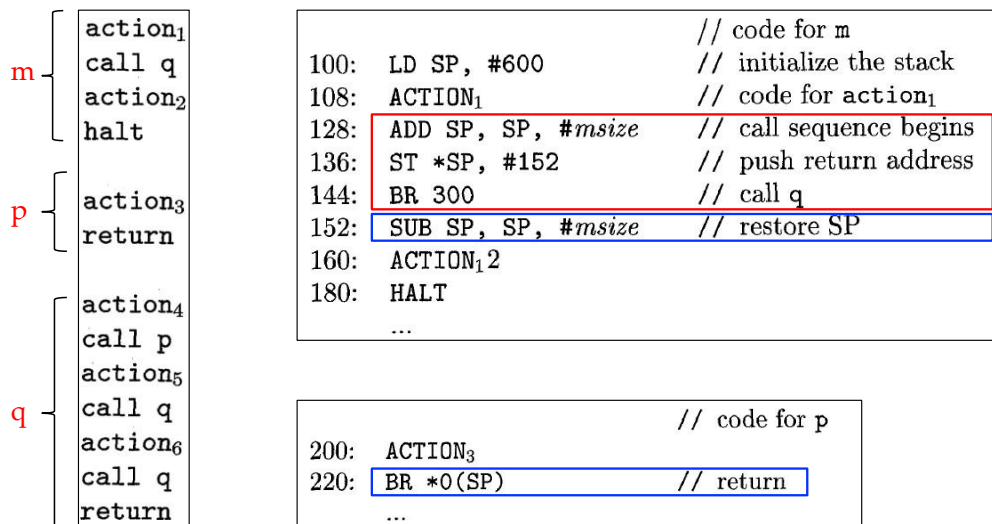
ST 指令中的运算分量#*here*+16 是跟随 BR 之后的指令的地址；它保存在 SP 指向的地址中。

- 返回序列由两部分组成。
 - 被调用的过程使用以下命令将控制权转移到返回地址：在 BR 指令中使用*0(SP)的原因是我们需要两级间接寻址：0(SP)是活动记录中第一个字所在的位置，而*0(SP)是保存在那里的返回地址。
 - 返回序列的第二部分位于调用者中，这个序列减少 SP 的值，从而将 SP 恢复为之前的值。也就是说，减法运算后 SP 指向调用者活动记录的开头

Example

Call sequence

Return sequence



• 例 8.4: 图 8.5 中的程序是上一章中的快速排序程序的抽象。过程 q 是递归的，因此 q 的同一时刻可能有多个活跃的 q 的活动记录。

• 假设过程 m、p 和 q 的活动记录的大小已分别确定为 msize、psize 和 qsize。每个活动记录中的第一个字将保存一个返回地址。我们任意假设这些过程的代码分别从地址 100、200 和 300 开始，栈从地址 600 开始。目标程序如图 8.6 所示。

• 我们假设 ACTION₄ 包含了一个条件跳转指令，跳转到 q 的返回代码序列开始地址 456；否则，递归过程 q 注定会永远调用自己。

• 令 msize、psize 和 qsize 分别为 20、40 和 60。地址 100 处的第一条指令将 SP 初始化为 600，即栈的起始地址。在控制权从 m 转移到 q 的前一刻，SP 中的值是 620（因为 msize 为 20）。随后，当 q 调用 p 时，在地址 320 处的指令将 SP 递增到 680，p 的活动记录从这里开始；当控制返回到 q 的时候，SP 恢复到 620。如果接下来的两个对 q 的递归调用立即返回，则此执行期间 SP 的最大值为 680。但是请注意，栈区中最后被使用的位置是 739，因为从位置 680 开始的 q 的活动记录总共有 60 字节。？

Example

m	action ₁	300: ACTION ₄ // code for q
	call q	
	action ₂	
	halt	
p	action ₃	320: ADD SP, SP, #qsize // contains a conditional jump to 456
	return	
q	action ₄	328: ST *SP, #344 // push return address
	call p	
	action ₅	
	call q	
	action ₆	
	call q	
	return	
		336: BR 200 // call p
		344: SUB SP, SP, #qsize
		352: ACTION ₅
		372: ADD SP, SP, #qsize
		380: BR *SP, #396 // push return address
		388: BR 300 // call q
		396: SUB SP, SP, #qsize
		404: ACTION ₆
		424: ADD SP, SP, #qsize
		432: ST *SP, #440 // push return address
		440: BR 300 // call q
		448: SUB SP, SP, #qsize
		456: BR *0(SP) // return
		...
		600: // stack starts here

Addresses in the Target Code

- How to generate code for procedure calls and returns?
 - Static allocation (静态分配)
 - Stack allocation (栈式分配)
- How to replace names in IR by code to access storage locations?

Run-Time Addresses for Names

- A **name** in a three-address statement corresponds to a symbol-table entry
- Statement $x = 0$
 - Suppose the symbol-table entry for x contains a relative address 12
 - If x is in a **statically allocated area** (i.e., **static**):
 - The effect of $x = 0$: `static[12] = 0`
 - Target code: `LD 112, #0` (suppose static area starts at address 100)
 - If x is in **stack** (in an activation record):
 - `LD 12(SP), #0`

• 存储分配策略和过程的活动记录中局部数据的布局决定了如何访问名字对应的内存位置。在第六章中，我们假设三地址语句中的名字实际上是指向该名字的符号表条目的指针。这种方法有一个显着的优点：它使编译器更加易于移植，因为即使将编译器移植到使用不同运行时组织的其他机器上，也不需要更改前端。另一方面，在生成中间代码的同时生成特定的访问步骤序列对于一个优化编译器也有极大的好处，因为它可以让优化器利用原本在简单的三地址语句中看不到的细节。

• 无论哪种情况，名字最终都必须被替换为访问存储位置的代码。因此，我们考虑简单三地址复制语句 $x = 0$ 的一些细节。

- 在处理过程中的声明之后，假设 x 的符号表条目包含 x 的相对地址 12。

- 例如，考虑 x 位于从地址 `static` 开始的静态分配区域中的情况。那么 x 的实际运行时地址是 `static + 12`。虽然编译器最终可以在编译时确定 `static + 12` 的值，但是在生成访问该名字的中间代码时，可能还不知道静态区域的位置。在这种情况下，生成“计算” `static + 12` 的三地址代码是有意义的。当然我们能要理解，该计算在程序运行之前就会完成：它或者在代

码生成阶段完成，或者由加载器完成。那么，赋值语句 $x = 0$ 被翻译成 `static[12] = 0`；如果静态区域从地址 100 开始，则该语句的目标代码是 `LD 112, #0`

- 如果 x 位于栈中（活动记录中），目标代码是 `LD 12(SP), #0`

Basic Block and Flow Graph

- Graph representation of intermediate code
 - Partition the intermediate code into **basic blocks**
 - The flow of control can only enter the basic block through its first instruction
 - Control will not leave the block, except possibly at the last instruction in the block (**no halting/branching in the middle**)
 - **Flow graphs**: basic blocks are **nodes** and the **edges** indicate which block can follow which other blocks
- Flow graphs form **the basis of code optimization**
 - They describe how control flows among basic blocks
 - We can know how values are defined and used

本节介绍中间代码的图形表示，即使图形不是由代码生成算法显式构造的，它也有助于讨论代码生成。代码生成受益于上下文。如果我们知道如何定义和使用值，我们就可以更好地进行寄存器分配，正如我们将在第 8.8 节中看到的那样。我们可以通过查看三地址语句序列来更好地进行指令选择，正如我们将在第 8.9 节中看到的那样。该表示的构造如下：

1. 将中间代码划分为基本块，这些基本块是连续三地址指令的最大序列，具有以下属性：
 - (a) 控制流只能通过基本块中的第一个指令进入该块。也就是说，没有跳转到块中间的转移指令。
 - (b) 除了基本块的最后一条指令，控制流在离开基本块之前不会停机或者跳转。
2. 基本块成为流图的节点，流图的边指示哪些块可以跟随哪些其他块。

从第 9 章开始，我们讨论流图上的转换，将原始中间代码转换为“优化后”的中间代码，而从中可以生成更好的目标代码。使用“优化后”的中间代码将其转换为机器代码将使用本章中的代码生成技术。

流程图是代码优化的基础，它们描述了控制如何在基本模块之间流动，我们可以知道值是如何定义和使用的。

Partitioning Three-Address Instructions into Basic Blocks

- **Input:** A sequence of three-address instructions
- **Output:** A list of basic blocks (each inst. is assigned to one block)
- **Method:**
 - Rules for finding *leader instructions* (首指令, the 1st instruction of a basic block)
 1. The *first instruction* in the entire intermediate code is a leader
 2. Any instruction that is the *target of a conditional/unconditional jump* is a leader
 3. Any instruction that *immediately follows a conditional/unconditional jump* is a leader
 - Then, for each leader, its basic block consists of *itself and all instructions up to but not including the next leader* or the end of the intermediate program

我们的第一项工作是将一系列三地址指令划分为基本块。我们从第一条指令开始一个新的基本块，并不断添加指令，直到我们遇到下一条指令的跳转、条件跳转或下一个指令前的标号为止。当没有跳转和标号的情况下，控制按顺序从一条指令进行到下一条指令。这个想法在以下算法中形式化地表示出来。

算法 8.5：将三地址指令划分为基本块。

输入：三地址指令序列。

输出：该序列的基本块列表，其中每条指令恰好被分配给一个基本块。

方法：首先，我们确定中间代码中的哪些指令是首指令，即某个基本块中的第一个指令。跟在中间程序末尾之后的指令不包含在首指令集合中。选择首指令的规则如下：

1. 中间代码中的第一条三地址指令是首指令。
2. 作为条件或无条件跳转指令的目标指令是一个首指令。
3. 紧跟在一个条件或无条件跳转之后的指令是一个首指令。

然后，每个首指令对应的基本块包括从它自己开始，直到下一个首指令（不含）或者中间程序的结尾指令之间的所有指令。

Basic Block Example

```
for i from 1 to 10 do
  for j from 1 to 10 do
    a[i, j] = 0.0;
  for i from 1 to 10 do
    a[i, i] = 1.0;
```

- **Leader instructions:**

- The first instruction (rule #1): 1
- Targets of jumps (rule #2): 3, 2, 13
- Instructions immediately following jumps (rule #3): 10, 12

- **Basic blocks**

- 1-1 2-2
- 3-9 10-11
- 12-12 13-17

```

→ 1) i = 1
→ 2) j = 1
→ 3) t1 = 10 * i
  4) t2 = t1 + j
  5) t3 = 8 * t2
  6) t4 = t3 - 88
  7) a[t4] = 0.0
  8) j = j + 1
  9) if j <= 10 goto (3)
→ 10) i = i + 1
  11) if i <= 10 goto (2)
→ 12) i = 1
→ 13) t5 = i - 1
  14) t6 = 88 * t5
  15) a[t6] = 1.0
  16) i = i + 1
  17) if i <= 10 goto (13)
```

图 8.7 中的中间代码将 10X10 矩阵 a 转换为单位矩阵。尽管该代码来自何处并不重要，但它可能是图 8.8 中伪代码的翻译。在生成中间代码时，我们假设每个实数值的数组元素占用 8 个字节，并且矩阵 a 以行优先形式存储。

首先，根据算法 8.5 的规则 (1)，指令 1 是一个首指令。为了找到其他的首指令，我们首先要找到跳转指令。在这个例子中，指令 9、11 和 17 处有 3 个跳转，都是有条件跳转指令。根据规则 (2)，这些跳转指令的目标是首指令；它们分别是指令 3、2 和 13。然后，根据规则 (3)，跟在一个跳转指令后的每条指令都是首指令；它们是指令 10 和 12。请注意，在此代码中，17 后面没有指令，但如果后面有代码，则第 18 条指令也将是首指令。

我们得出结论，指令 1、2、3、10、12 和 13 是首指令。每个首指令对应的基本块包含了从其自身到下一个首指令之前的所有指令。因此，指令 1 的基本块就是指令 1，指令 2 的基本块是指令 2。然而，首指令 3 的基本块包含了从指令 3 到 9 的所有指令。指令 10 的基本块是 10 和 11；指令 12 的基本块只包含指令 12，而指令 13 的基本块是指令 13 到 17。

Next-Use Information (后续使用信息)

- The *use* of a name in a three-address statement
 - Suppose a statement i assigns a value to x . If statement j has x as an operand, and control can flow from i to j along a path that has no intervening assignments to x , then we say j *uses the value of x computed at i*
 - We further say that x is *live* (活跃) at statement i
 - *Not live*: the *value* computed will not be used, not the variable will not be used
- **Code generation can benefit from such information**
 - If the value of a variable that is currently in a register will never be referenced subsequently, we can assign the register to another variable

在一个三地址语句中对一个名字的使用定义如下。

- 假设三地址语句 i 给 x 赋了一个值。如果语句 j 将 x 作为运算分量，并且控制可以沿着没有中间对 x 赋值的路径从语句 i 流向 j ，则我们说语句 j 使用在语句 i 计算的 x 值。
- 我们进一步说 x 在语句 i 处处于活动状态。

知道一个变量的值接下来会在何时使用对于生成良好的代码至关重要。如果当前位于寄存器中的变量的值随后永远不会被引用，则可以将该寄存器分配给另一个变量。

Determine Liveness and Next-Use Information for Each Statement

- **Input:** A basic block B . Assume that the symbol table initially shows all nontemporary variables in B as being **live on exit**
- **Output:** At each statement $i: x = y \text{ op } z$, we attach to i the **liveness and next-use information** of x , y , and z . Here, **we do not consider uses outside the basic block**
- **Method:**
 - **Scan backwards from the last statement in B .** At each statement $i: x = y \text{ op } z$, do the following (the order matters as x maybe y or z):
 1. Attach to i the information currently in the symbol table regarding the next use and liveness of x , y , and z
 2. In the symbol table, set x to “not live” and “no next use”
 3. In the symbol table, set y and z to “live” and their next uses to i

对于每个类似于 $x = y + z$ 的三地址语句，我们希望确定对 x 、 y 和 z 的下一使用是什么。我们确定活跃性和后续使用信息的算法对每个基本块进行一次反向遍历。我们将得到的信息存储在符号表中。（在扫描过程中通过为每个名字建立相应的下次引用信息链何额活跃变量信息链，收集各名字在基本块内的下次引用信息和活跃信息）。活跃信息用于代码优化时全局数据流分析。）如算法 8.5 所示，我们可以轻松地扫描三地址语句流来找到各个基本块的结尾。由于过程可以有任意的副作用（比如修改某个外部变量的值），为了方便起见，我们假设每个过程调用指令是一个新的基本块的入口。

对于一个基本块中的每一个语句确定活跃性与后续使用信息。

输入：一个三地址语句的基本块 B 。我们假设最初符号表显示 B 中的所有非临时变量都是活跃的。（如果产生中间代码的算法允许某些临时变量在基本块之外引用，也假定这些临时变量在出口处是活跃的。）

输出：对于 B 中的每个语句 $i: x = y + z$ 中，我们将 x 、 y 和 z 的活跃信息以及后续使用信息关联到 i 。这里，我们不关心考虑包含本三地址语句的基本块之外的使用。

为了计算名字的下次引用信息，需要对符号表进行扩充，增加记录下次引用信息和活跃信息的域，同时对保存三地址语句的结构进行扩充，使之可以记录语句中出现的每个名字的下次引用信息和活跃信息

方法：我们从 B 中的最后一个语句开始，反向扫描到 B 的开始处。对于 B 中的每个语句 $i: x = y + z$ ，我们执行以下操作：

1. 把在当前符号表中找到的有关 x 、 y 和 z 的当前后续使用和活跃信息与语句 i 关联起来。

2. 在符号表中，将 x 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 x 被重新计算了，之前的值被更改）

3. 在符号表中，将 y 和 z 设置为“活跃”，并把它们的下一次使用设置为语句 i 。

这里，我们使用 $+$ 作为代表任意运算符的符号。如果三地址语句 i 形如 $x = +y$ 或 $x = y$ ，那么处理步骤依然和上面相同，只是忽略了对 z 的处理。请注意，步骤 (2) 和 (3) 的顺序不能颠倒，因为 x 可能是 y 或 z 。

通过应用此算法，基本块中所有名字的下次引用信息都被收集到，并且保存于符号表中和三地址语句上。如果一个变量在基本块中被引用，则由该变量在符号表中的下次引用信息，以及附加在各三地址语句 i 上的信息，可以从前到后依次指示出该名字在基本块中的各个引用所在的位置。

Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- i, j, a are not temporary names generated by compilers. Initially, only they are live on exit

Name	Liveness	Next-use
i	Live	
j	Live	
a	Live	
$t1$	Not live	
$t2$	Not live	
$t3$	Not live	
$t4$	Not live	

为了计算名字的下次引用信息，需要对符号表进行扩充，增加记录下次引用信息和活跃信息的域。

我们假设最初符号表显示 B 中的所有非临时变量都是活跃的。

Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
→ 8) j = j + 1

```

- **Analyzing line 8):** attach the following information to 8)

Name	Liveness	Next-use
<i>j</i>	Live*	

* **Live at statement 8 (block exit):** after executing statement 8, the value of *j* will be used by subsequent statements

1. 把在当前符号表中找到的有关 *x*、*y* 和 *z* 的当前后续使用和活跃信息与语句 8 关联起来。(j)

Running Example

```

3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
→ 8) j = j + 1

```

- **Analyzing line 8):** update symbol table*

Name	Liveness	Next-use
<i>i</i>	Live	
<i>j</i>	Live	8
<i>a</i>	Live	
<i>t1</i>	Not live	
<i>t2</i>	Not live	
<i>t3</i>	Not live	
<i>t4</i>	Not live	

* The updated information will be passed and attached to the previous statements

2. 在符号表中，将 j 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 x 被重新计算了，之前的值被更改）
3. 在符号表中，将 j 设置为“活跃”，并把它们的下一次使用设置为语句 8。

Running Example

```
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
→ 7)  a[t4] = 0.0
8)  j = j + 1
```

- **Analyzing line 7):** attach the following information to 7)

Name	Liveness	Next-use
a	Live	
$t4$	Not live	

- Fall 2024BUPT Compilers32
1. 把在当前符号表中找到的有关 x 、 y 和 z 的当前后续使用和活跃信息与语句 7 关联起来。

Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
→ 7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 7):** update symbol table

Name	Liveness	Next-use
<i>i</i>	Live	
<i>j</i>	Live	8
<i>a</i>	Live	7
<i>t1</i>	Not live	
<i>t2</i>	Not live	
<i>t3</i>	Not live	
<i>t4</i>	Live	7

2. 在符号表中，将 *a* 和 *t* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将 *a* 和 *t4* 设置为“活跃”，并把它们的下一次使用设置为语句 7。

Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
→ 6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 6):** attach the following information to 6)

Name	Liveness	Next-use
<i>t3</i>	Not live	
<i>t4</i>	Live	7

1. 把在当前符号表中找到的有关 *t4*、*t3* 的当前后续使用和活跃信息与语句 6 关联起来。

Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
→ 6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 6):** update symbol table

Name	Liveness	Next-use
<i>i</i>	Live	
<i>j</i>	Live	8
<i>a</i>	Live	7
<i>t1</i>	Not live	
<i>t2</i>	Not live	
<i>t3</i>	Live	6
<i>t4</i>	Not live	

2. 在符号表中，将 t_4 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将 t_3 设置为“活跃”，并把它们的下一次使用设置为语句 6。

Running Example

```
3)  t1 = 10 * i
4)  t2 = t1 + j
→ 5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
```

- **Analyzing line 5):** attach the following information to 5)

Name	Liveness	Next-use
t_2	Not live	
t_3	Live	6

- Fall 2024BUPT Compilers36
1. 把在当前符号表中找到的有关 t_3 、 t_2 的当前后续使用和活跃信息与语句 5 关联起来。

Running Example

```
3) t1 = 10 * i
4) t2 = t1 + j
→ 5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
```

- **Analyzing line 5):** update symbol table

Name	Liveness	Next-use
<i>i</i>	Live	
<i>j</i>	Live	8
<i>a</i>	Live	7
<i>t1</i>	Not live	
<i>t2</i>	Live	5
<i>t3</i>	Not live	
<i>t4</i>	Not live	

2. 在符号表中，将 *t3* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 *t3* 被重新计算了，之前的值被更改）
3. 在符号表中，将 *t2* 设置为“活跃”，并把它们的下一次使用设置为语句 5。

Running Example

```
3) t1 = 10 * i  
→ 4) t2 = t1 + j  
5) t3 = 8 * t2  
6) t4 = t3 - 88  
7) a[t4] = 0.0  
8) j = j + 1
```

- **Analyzing line 4):** attach the following information to 4)

Name	Liveness	Next-use
<i>j</i>	Live	8
<i>t1</i>	Not live	
<i>t2</i>	Live	5

1. 把在当前符号表中找到的有关 *t2*、*t1* 和 *j* 的当前后续使用和活跃信息与语句 4 关联起来。

Running Example

```
3) t1 = 10 * i  
→ 4) t2 = t1 + j  
5) t3 = 8 * t2  
6) t4 = t3 - 88  
7) a[t4] = 0.0  
8) j = j + 1
```

- **Analyzing line 4):** update symbol table

Name	Liveness	Next-use
<i>i</i>	Live	
<i>j</i>	Live	4
<i>a</i>	Live	7
<i>t1</i>	Live	4
<i>t2</i>	Not live	
<i>t3</i>	Not live	
<i>t4</i>	Not live	

2. 在符号表中，将 $t2$ 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为? 被重新计算了，之前的值被更改）
3. 在符号表中，将 $t1$ 和 j 设置为“活跃”，并把它们的下一次使用设置为语句 4。

Running Example

→ 3) $t1 = 10 * i$
 4) $t2 = t1 + j$
 5) $t3 = 8 * t2$
 6) $t4 = t3 - 88$
 7) $a[t4] = 0.0$
 8) $j = j + 1$

- **Analyzing line 3):** attach the following information to 3)

Name	Liveness	Next-use
i	Live	
$t1$	Live	4

1. 把在当前符号表中找到的有关 $t1$ 、 i 和 j 的当前后续使用和活跃信息与语句 3 关联起来。

Running Example

→ 3) $t1 = 10 * i$
4) $t2 = t1 + j$
5) $t3 = 8 * t2$
6) $t4 = t3 - 88$
7) $a[t4] = 0.0$
8) $j = j + 1$

- **Analyzing line 3):** update symbol table

Name	Liveness	Next-use
<i>i</i>	Live	3
<i>j</i>	Live	4
<i>a</i>	Live	7
<i>t1</i>	Not live	
<i>t2</i>	Not live	
<i>t3</i>	Not live	
<i>t4</i>	Not live	

2. 在符号表中，将 *t1* 的活跃信息和下次引用信息分别设置为“不活跃”和“无后续使用”。（因为 *t1* 被重新计算了，之前的值被更改）
3. 在符号表中，将 *i* 设置为“活跃”，并把它们的下一次使用设置为语句 3。

Flow Graphs

- The nodes of the flow graphs are the basic blocks
 - There is an edge from block B to block C **iff** it is possible for the first instruction in C to immediately follow the last instruction in B
- Possible reasons for the introduction of edges ($B \rightarrow C$)
 - **Case 1:** There is a (un)conditional jump from the end of B to the beginning of C
 - **Case 2:** C immediately follows B in the original three-address code, and B does not end in an unconditional jump
 - We say that B is the *predecessor* (前驱) of C and C is the *successor* (后继) of B
- Two special nodes: *entry* (入口结点) and *exit* (出口结点)
 - They do not correspond to executable intermediate instructions
 - There is an edge from entry node to the first executable node of the flow graph; there is an edge to the exit from any basic block that could be executed last

一旦中间代码程序被划分为基本块，我们就可以通过流图来表示它们之间的控制流。流图的节点是基本块。当且仅当块 C 中的第一条指令可能紧跟在块 B 中的最后一条指令之后时，才存在从块 B 到块 C 的边沿。存在这样的一条边的原因有两种：

Case1:有一个从 B 的末尾到 C 的开头的有条件或无条件跳转。

Case2:按照原来的三地址指令序列中的顺序， C 紧跟在 B 之后，且 B 的结尾不存在无条件跳转语句。

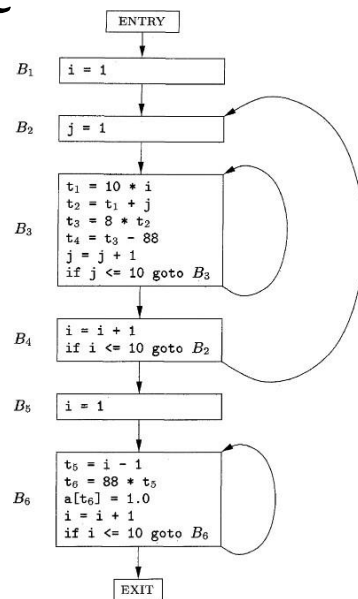
- 我们说 B 是 C 的前驱，而 C 是 B 的后继。

通常会添加两个分别称为入口和出口的结点

- 它们不和任何可执行的中间指令对应。
- 从入口到流图的第一个可执行节点（即包含了中间代码的第一条指令的基本块）有一条边。从任何包含了可能是程序的最后执行指令的基本块到出口有一条边。如果程序的最后指令不是一个无条件跳转，则包含了程序最后指令的块是出口结点的一个前驱。但任何包含了跳转到程序之外的跳转指令的基本块也是出口结点的前驱。

Flow Graph Example

- Edges introduced because of jumps (Case 1)
 - $B_3 \rightarrow B_3$
 - $B_4 \rightarrow B_2$
 - $B_6 \rightarrow B_6$
- The other edges belong to “Case 2”



例 8.8: 例 8.6 (10X10 矩阵转成单位阵) 中构造的一组基本块可以生成图 8.9 的流图。入口点指向基本块 B1, 因为 B1 包含程序的第一条指令。B1 的唯一后继者是 B2, 因为 B1 的结尾不是一个无条件跳指令, 且 B2 的首指令紧跟在 B1 的结尾指令之后。

基本块 B3 有两个后继。其中的一个是它本身, 因为 B3 的首指令 (指令 3) 是 B3 结尾处的条件跳转指令 (指令 9) 的目标。另一个后继是 B4, 因为控制可以穿越 B3 结尾处的条件跳转指令而到达 B4 的首指令。

只有 B6 指向流图的出口结点, 因为到达紧跟在流图对应的程序之后的代码的唯一方法是穿越 B6 结尾处的条件跳转指令。

首先, 从图 8.9 中注意到, 在流程图中, 把到达指令的序号或标号的跳转指令替换为到达基本块的跳转, 这么做是很正常的。回想一下, 每个条件或无条件跳转都是到某个基本块的首指令, 而现在这些跳转指令指向了相应的基本块。这么做的原因是, 在构建了流图之后, 经常会对多个基本块中的指令进行实质性更改。如果跳转的目标是指令, 我们将不得不在每次改变了某个目标指令之后修正跳转指令的目标。

流图就是通常的图, 它可以用任何表示图的数据结构来表示。结点 (基本块) 的内容需要有它们自己的表示方式。我们可能会用一个指向该基本块在三地址指令组中的首指令

的指针，以及基本块的指令数量或一个指向结尾指令的指针来表示结点的内容。然而，由于我们可能经常更改基本块中的指令数量，所以为每个基本块创建一个指令链接列表可能会更有效。

Loops (循环)

- Programs often spend most of the time in executing loops. It is important for compilers to generate efficient code for loops.
- **Definition of loops:**
 - A loop L is a set of nodes in the flow graph
 - L contains a node e called the *loop entry* (循环入口)
 - No node in L except e has a predecessor outside L . That is, every path from the entry of the entire flow graph to any node in L goes through e .*
 - Every node in L has a nonempty path, completely within L , to e

* We say e dominates the other nodes in L

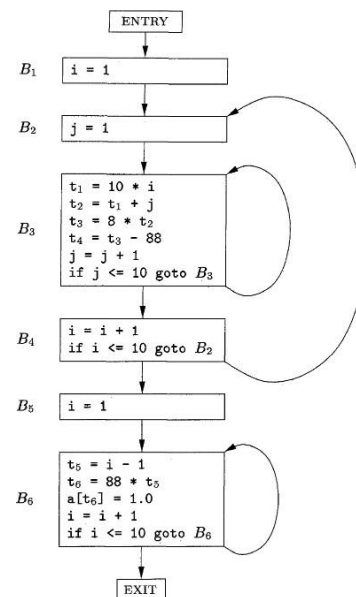
像 while 语句、do-while 语句和 for 语句这样的程序语言结构自然地把循环引入到程序中。由于实际上每个程序的大部分时间都在执行其循环操作，因此编译器为循环操作生成良好的代码尤为重要。许多代码转换最终取决于流图中“循环”的识别。如果下列条件成立，我们就是流图中的一个结点集合 L 是一个循环，

1) 在 L 中有一个被称为循环入口的结点，它是唯一的其前驱可能在 L 之外的结点。也就是说，从整个流图的入口结点开始到 L 中的任何结点的路径都必然经过循环入口结点，并且这循环入口结点不是整个流图的入口结点本身。

2) L 中的每个结点都有一个到达 L 的入口结点的非空路径，并且该路径全部都在 L 中。

Loop Examples

- $\{B_3\}$
- $\{B_2, B_3, B_4\}$
 - B_2 is the loop entry
 - B_1, B_5, B_6 are not in the loop
 - There is a path from ENTRY to B_1 that does not go through B_2
 - B_5 and B_6 have no nonempty paths to B_2
- $\{B_6\}$



示例 8.9：图 8.9 的流图具有三个循环：

1. B_3 本身。
2. $\{B_2; B_3; B_4\}$ 。

$L = \{B_2; B_3; B_4\}$ 的循环入口结点是 B_2 。请注意，这三个节点中，只有 B_2 有一个不在 L 中的前驱 B_1 。此外，这三个结点中都有在 L 中且到达 B_2 的非空路径。比如，从 B_2 开始就有路径 $B_2 \rightarrow B_3 \rightarrow B_4 \rightarrow B_2$ 。

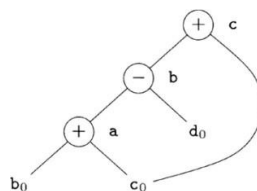
3. B_6 本身。

B_3 和 B_6 是单个节点，这些结点都有到其自身的边。例如， B_3 形成一个以 B_3 本身为入口的循环。请注意，循环的最后一个要求是从 B_3 到其自身有一条非空路径。因此，像 B_2 这样的单个节点（它没有一条 $B_2 \rightarrow B_2$ 的边）不是循环，因为没有从 B_2 到其自身，且在集合 $\{B_2\}$ 中的非空路径。

The DAG Representation of Basic Blocks

- The **DAG (directed acyclic graph)** of a basic block depicts the relationships among the values of all variables in a basic block when it executes (**data dependence**)
- **DAG enables several code-improving transformations:**
 - Eliminate local common subexpressions (**局部公共子表达式**)
 - Eliminate dead code (**死代码**)
 - Apply algebraic laws (**代数恒等式**) to reorder operands of instructions

$a = b + c$
 $b = a - d$
 $c = b + c$



Fall 2024

BUPT Compilers

47

许多重要的局部优化技术都是从将基本块转换为 DAG（有向无环图）开始的。在第 6.1.1 节中，我们介绍了 用于表示简单表达式的 DAG。这一想法自然地延伸到在一个基本块内创建的表达式的集合。

基本块的 DAG 表示允许我们对块表示的代码执行一些转换，以改进代码的质量。

- 我们可以消除局部公共子表达式，所谓公共子表达式就是重复计算一个已经计算过的值的指令。
- 我们可以消除死代码，即计算得到的值不会被使用的指令。
- 我们可以对相互独立的语句重新排序；这种重新排序可以减少临时值需要保存在寄存器中的时间。
- 我们可以应用代数定律对三地址指令的运算分量重新排序，这么做有时可以简化计算过程。

Constructing DAG's

$a = b + c$
 $b = a - d$
 $c = b + c$

a_0 b_0 c_0 d_0

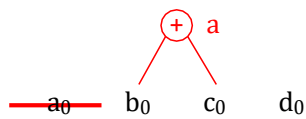
Step 1: Create a node for each of the initial values of the variables in the basic block

我们为基本块构造一个 DAG，如下所示：

1. 对于基本块中出现的每个变量有一个对应的 DAG 的结点表示其初始值。

Constructing DAG's

$a = b + c$
 $b = a - d$
 $c = b + c$



The variable a get a new value.
Old value a_0 is never used (killed)

Step 2: Process $a = b + c$

- Create a node N for the statement
- Label it with $+$
- Attach a to N
- The children of N are those nodes corresponding to the **last definitions** of b and c

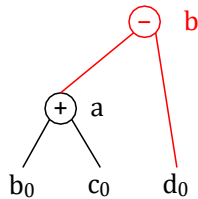
2. 基本块内的每个语句 s 都有一个相关联的结点 N 。 N 的子结点是基本块中的其他语句的对应结点。这些语句是在 s 之前、最后一个对所使用的某个运算分量进行定值的语句。

3. 结点 N 的标号是 s 中的运算符；同时还有一组变量被关联到 N ，表示 s 是在此基本块内最组晚对这些变量进行定值的语句。

4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

Constructing DAG's

$a = b + c$
 $b = a - d$
 $c = b + c$



Step 3: Process $b = a - d$

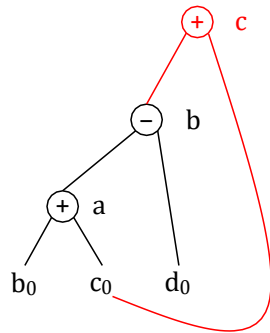
- Create a node N for the statement
- Label it with $-$
- Attach b to N
- The children of N are those nodes corresponding to the **last definitions** of a and d

我们为基本块构造一个 DAG，如下所示：

1. 对于基本块中出现的每个变量有一个对应的 DAG 的结点表示其初始值。
2. 基本块内的每个语句 s 都有一个相关联的结点 N 。 N 的子结点是基本块中的其他语句的对应结点。这些语句是在 s 之前、最后一个对所使用的某个运算分量进行定值的语句。
3. 结点 N 的标号是 s 中的运算符；同时还有一组变量被关联到 N ，表示 s 是在此基本块内最组晚对这些变量进行定值的语句。
4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

Constructing DAG's

$a = b + c$
 $b = a - d$
 $c = b + c$



Step 4: Process $c = b + c$

- Create a node N for the statement
- Label it with $+$
- Attach c to N
- The children of N are those nodes corresponding to the **last definitions** of b and c

2. 基本块内的每个语句 s 都有一个相关联的结点 N 。 N 的子结点是基本块中的其他语句的对应结点。这些语句是在 s 之前、最后一个对所使用的某个运算分量进行定值的语句。

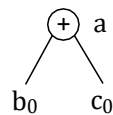
3. 结点 N 的标号是 s 中的运算符；同时还有一组变量被关联到 N ，表示 s 是在此基本块内最组晚对这些变量进行定值的语句。

4. 某些结点被指明为输出结点。这些结点的变量在基本块的出口处活跃。也就是说，这些变量的值可以稍后在流图的另一个块中使用。这些“活跃变量”的计算是全局数据流分析的问题，在第 9.2.5 节中讨论。

Finding Local Common Subexpression

- When creating a node M , check if there exists a node N , which has the same operator and children nodes (order matters) with M
- If such a node N exists, we do not create the node M but simply use N to represent M

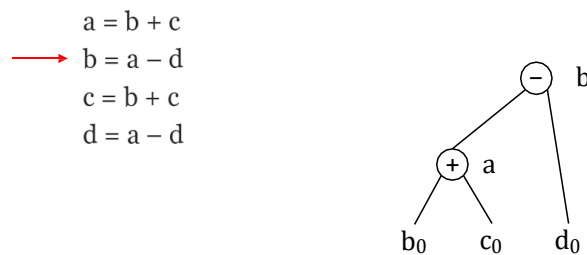
→ $a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$



检测公共子表达式的方法是这样的，当一个新的节点 M 被加入到 DAG 时，检测是否存在一个结点 N ，它和 M 具有相同运算符和子节点，且子节点的顺序相同。如果是这样， N 计算的值与 M 计算的的值是一样的，因此，可以用 N 替换 M 。该技术在第 6.1.1 节中作为检测公共子表达式的“值编码”方法引入。

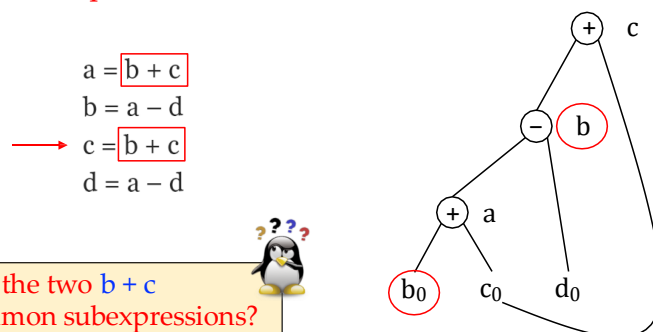
Finding Local Common Subexpression

- When creating a node M , check if there exists a node N , which has the same operator and children nodes (order matters) with M
- If such a node N exists, we do not create the node M but simply use N to represent M



Finding Local Common Subexpression

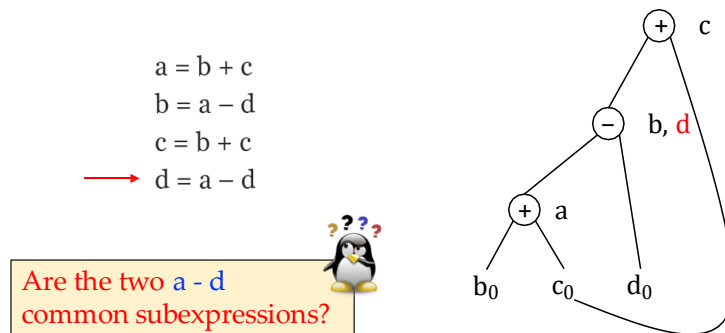
- When creating a node M , check if there exists a node N , which has the same operator and children nodes (order matters) with M
- If such a node N exists, we do not create the node M but simply use N to represent M



当我们为第三条语句 $c = b + c$ 构造结点时，我们知道 $b + c$ 中使用的 b 指的是图 8.12 中标号为 $-$ 的节点。因为这个结点是 b 的最近的定值。因此，我们不会混淆语句 1 和语句 3 所计算的值。

Finding Local Common Subexpression

- When creating a node M , **check if there exists a node N** , which has the same operator and children nodes (order matters) with M
- If such a node N exists, we do not create the node M but simply **use N to represent M**



然而，对应于第四条语句 $d = a - d$ 的结点的运算符是 $-$ ，且它的子结点是标记有变量 a 和 d_0 的结点。由于运算符和子结点都与语句 2 对应的结点相同，我们不需要创建此结点，而是将 d 添加到这个标记为 $-$ 的结点的定值变量表中。

由于图 8.12 的 DAG 中只有三个非叶节点，看起来示例 8.10 中的基本块可以替换为一个只有三个语句的基本块。事实上，如果 b 在退出块时不活跃，那么我们不需要计算该变量 b ，并且可以使用 d 来存放由图 8.12 中的标号为 $-$ 的结点所代表的值。这个基本块就变成了

$a = b + c$

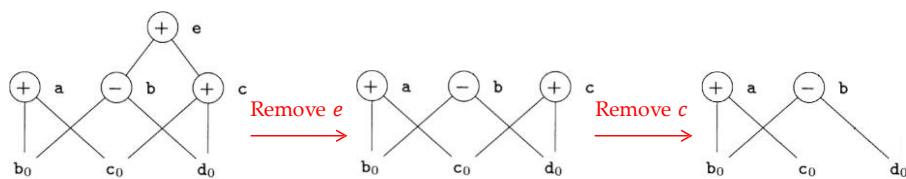
$d = a - d$

$c = d + c$

然而，如果 b 和 d 在出口处活跃，则必须使用第四条语句将值从一个复制到另一个。

Dead Code Elimination

- We can delete from a DAG any **root** (node without ancestors) that has **no live variables** attached
- **Repeatedly applying the above transformation** will remove all nodes corresponding to dead code
- Suppose in the example below, **c and e are not live** (they have no next uses), but **a and b are live**



对应于死码消除的 DAG 操作可以按如下方式实现。

我们从一个 DAG 上删除所有没有附加活跃变量的根结点（没有祖先的节点）。

重复应用此处理过程，就可从 DAG 中删除与死代码对应的所有节点。

例 8.12: 如果在图 8.13 中, a 和 b 是活跃变量, 而 c 和 e 不是, 我们可以立即删除标记为 e 的根结点。然后, 标记为 c 的结点就变成根节点, 也可以被删除。标有 a 和 b 的根结点被保留下来, 因为它们各自都附加了活跃变量。

The Use of Algebraic Identities

- **Eliminate computations (消除计算步骤)** from a basic block
 - $x + 0 = 0 + x = x$ $x - 0 = x$
 - $x \times 1 = 1 \times x = x$ $x / 1 = x$
- **Reduction in strength (降低计算强度)**, replacing a more **expensive** operator by a **cheaper** one
 - $2 \times x = x + x$ $x / 2 = x \times 0.5$
- **Constant folding (常量合并)**
 - $2 \times 3.14 = 6.28$

We can implement such optimizations by looking for patterns in a DAG

代数恒等式代表了基本块上另一类重要的优化。例如，我们可以应用算术恒等式，例如消除基本块中的计算步骤。

另一类代数优化包括局部强度削弱，即用代价低的运算替代代价高的运算，如下所示：

第三类相关优化是常量合并。使用这种方法时，我们在编译时计算常量表达式，并把此常量表达式替换为求出的值。因此，表达式 2×3.14 将被 6.28 替换。实际中由于程序中频繁使用符号常量，所有会出现许多常量表达式。

Code Generator

- **Generate machine instructions from three-address code**
 - Assume there is exactly one machine instruction for each operator
 - Assume some registers are available to hold the values used in a basic block
- **Primary goal:** avoid generating unnecessary loads and stores, i.e., making the best use of registers
- **Four principal uses of registers:**
 1. Hold the operands to perform operations
 2. Hold temporaries
 3. Hold global values
 4. Help with run-time storage management (e.g., holding stack pointers)

在本节中，我们将考虑一种为单个基本块生成代码的算法。它依次考虑每个三地址指令，并跟踪哪些寄存器中有哪些值

基本目标是：避免生成不必要的加载和存储指令。

代码生成过程中的主要问题之一是决定如何最大程度地使用寄存器。寄存器有四种主要用途：

- 在大多数机器体系结构中，执行一个运算时该运算的部分或全部运算分量必须位于寄存器中。
- 寄存器很适合做临时存储器，即在计算较大表达式时保存其子表达式结果，或者更一般地说，寄存器适合用于保存仅在单个基本块中使用的变量的值。
- 寄存器用于保存在一个基本块中计算并在其他块中使用的（全局）值，例如，循环下标的值，每次循环都对该值做增量运算，并在循环体内多次被使用。
- 寄存器通常用于帮助运行时存储管理，例如，管理运行时堆栈，包括堆栈指针的维护以及堆栈的顶部元素也可能被存放在寄存器中。

这些都是相互竞争的需求，因为可用的寄存器数量是有限的。

Code Generation Algorithm Overview

- The basic process:

1. **Load:** Considers each three-address instruction in turn and decides what loads are necessary to get the needed operands into registers
2. **Computation:** After generating the loads, it generates the operation
3. **Store:** Then, if there is a need to store the result into a memory location, it also generates the store instruction

Only generates loads when necessary

Try not to overwrite the register whose value is still of use

本节中的算法假设有一组寄存器可用于保存块内使用的值。通常，这组寄存器不包括机器的所有寄存器，因为一些寄存器是专门为全局变量和管理堆栈保留的。我们假设基本块已经通过诸如公共子表达式合并之类的转换而被转换成我们希望的三地址指令序列。我们进一步假设对于每个运算符，有且只有一个对应的机器指令。这个指令对存放在寄存器中的所需的运算分量进行运算，并将结果保留存在寄存器中。机器指令的形式为

LD reg, mem

ST mem, reg

OP reg,reg,reg

我们的代码生成算法依次考虑每个三地址指令，

- 并决定需要哪些加载指令来把所需运算加载进寄存器。
- 在生成加载指令后，它会生成运算代码。
- 然后，如果需要将结果存储到内存位置，它还会生成相应的保存指令。

Code Generation Algorithm Overview

- Two important data structures

1. **Register descriptor (寄存器描述符)**: For each available register, keeping track of the variable names whose current value is in that register
2. **Address descriptor (地址描述符)**: For each program variable, keeping track of the locations where the current value of that variable can be found
 - A **location** may be a register, a memory address, a stack location

为了做出这些必要的决策，我们需要一个数据结构来说明哪些程序变量的值当前被存放在哪个或哪些寄存器里面。我们还需要知道给定变量的内存位置上的值是否就是该变量的正确值，因为该变量的新值可能已在寄存器中计算出来但尚未存放到内存中。这个数据结构具有以下描述符：

1. 对于每个可用寄存器，寄存器描述符跟踪有哪些变量的当前值存放在此寄存器中。由于我们将考虑那些用于存放在一个基本块内的局部值的寄存器，因此我们假设最初所有寄存器描述符都是空的。随着代码生成的进行，每个寄存器将保存零个或多个变量名字的值。

2. 对于每个程序变量，地址描述符会跟踪记录在哪个或哪些位置上可以找到该变量的当前值。该位置可能是寄存器、内存地址、堆栈位置或其中多个位置的组成的一个集合。该信息可以存储在该变量名对应的符号表条目中。

Algorithm Details (1)

- An essential part of the algorithm: *getReg(I)*
 - Select registers for each memory location associated with the three-address instruction *I*, according to the register and address descriptors, and data-flow info (e.g., variables live on block exit)

Obviously, how *getReg()* selects registers affects the quality of the generated machine code

该算法的一个重要部分是函数 `getReg(I)`，它为每个与三地址指令 *I* 有关的内存位置选择寄存器。函数 `getReg` 可以访问基本块的所有变量对应的寄存器和地址描述符，并且还可以访问某些有用的数据流信息，例如哪些变量在基本块出口处活跃。我们将在介绍基本算法之后讨论 `getReg`。虽然我们不知道总共有多少寄存器可用于存放基本块的局部数据，因此假设有足够的寄存器，使得在把值存放回内存，释放了所有可用寄存器之后，空闲的寄存器足以完成任何三地址运算。

Algorithm Details (2)

- For a three-address instruction $x = y + z$ (here, $+$ is a **generic operator**), do the following:
 1. Use `getReg()` to **select registers** R_x , R_y , and R_z for x , y , and z
 2. If y is not in R_y , according to the **register descriptor** for R_y , then generate an instruction
 - `LD  $R_y$ ,  $y'$` // y' is a **mem loc** for y according to y' 's **address descriptor**
 3. Similar to step 2, generate `LD  $R_z$ ,  $z'$` if necessary
 4. Generate instruction `ADD  $R_x$ ,  $R_y$ ,  $R_z$`

在一个形如 $x = y + z$ 的三地址指令中，我们将 $+$ 视为一般的运算符，将 `ADD` 视为等价的机器指令。因此，我们没利用 $+$ 的交换性。这样，当我们实现该运算时， y 的值必须位于 `ADD` 指令中提到的第二个寄存器中，而不是第三个寄存器中。可以按照下面的方式来改造算法：只要 $+$ 是满足交换律的运算符，算法同时为 $x = y + z$ 和 $x = z + y$ 生成代码，然后再选择一个比较好的代码序列。

运算的机器指令

对于三地址指令（例如 $x = y + z$ ），请执行以下操作：

1. 使用 `getReg( $x = y + z$ )` 选择 x 、 y 和 z 的寄存器。将这些称为 R_x 、 R_y 和 R_z 。
2. 如果 y 不在 R_y 中（根据 R_y 的寄存器描述符），则生成指令 `LD  $R_y$ ;  $y'$` ，其中 y' 是存放 y 的内存位置之一（ y' 可根据 y 的地址描述符得到）。
3. 类似地，如果 z 不在 R_z 中，则生成指令 `LD  $R_z$ ;  $z'$` ，其中 z' 是存放 z 的位置。
4. 生成指令 `ADD  $R_x$ ;  $R_y$ ;  $R_z$` 。

Algorithm Details (3)

- For a copy instruction $x = y$, do the following:
 - We assume *getReg()* will always select the same registers for x and y
 - If y is not in that register R_y , then generate an instruction $LD\ R_y, y$
- Ending a basic block
 - For temporary variables used within the block, when the block ends, we can forget about their value and assume their register is empty
 - If a variable is live on block exit (or if we don't know the liveness), generate $ST\ x, R_x$ for each x whose address descriptor does not say that its current value is in the memory location for x (the value in register is newer than that in memory)

复制语句的机器指令

$x = y$ 形式的三地址复制语句是一个重要的特例。我们假设 *getReg* 总是为 x 和 y 选择相同的寄存器。如果 y 尚未在该寄存器 R_y 中，则生成机器指令 $LD\ R_y ; y$ 。如果 y 已经在 R_y 中，我们什么也不做。我们只需调整 R_y 的寄存器描述符，表明 R_y 中也存放了 x 的值。

基本块的收尾工作

我们描述算法时表明，在代码结束的时候，基本块中使用的变量可能仅存放在某个寄存器中。如果这个变量是一个仅在块内使用的临时变量，那就没问题；当块结束时，我们可以忘记这些临时变量的值并假设其寄存器为空。但是，如果变量在块出口处活跃，或者如果我们不知道哪些变量在出口处活跃，那么我们需要假设稍后还会用到该变量的值。在这种情况下，对于每个变量 x ，如它的地址描述符表明它的值没有存放在 x 的内存位置上，我们必须生成指令 $ST\ x; R$ ，其中 R 是在基本块的结尾处存放 x 值的寄存器。

Algorithm Details (4)

Update register and address descriptors with four rules:

1. For the instruction `LD R, x`:
 - a) Change the **register descriptor** for *R* so it holds only *x*
 - b) Change the **address descriptor** for *x* by adding *R* as an additional location
2. For the instruction `ST x, R`:
 - a) Change the **address descriptor** for *x* to include its own memory location*

* Values in registers cannot be older than those in memory.

管理寄存器和地址描述符

当代码生成算法生成加载、存储和其他机器指令时，它必须更新寄存器和地址描述符。规则如下：

1. 对于指令 `LD R; x`
 - (a) 更改寄存器 *R* 的寄存器描述符，使其仅包含 *x*。
 - (b) 更改 *x* 的地址描述符，把寄存器 *R* 作为新增位置加入到 *x* 的位置集合中。
 - (c) 从任何不同于 *x* 的变量的地址描述符中删除 *R*。
2. 对于指令 `ST x; R`，更改 *x* 的地址描述符使之包含其自己的内存位置。

Algorithm Details (5)

Update register and address descriptors with four rules:

3. For an operation such as `ADD  $R_x$ ,  $R_y$ ,  $R_z$` for implementing a three-address instruction $x = y + z$:
 - a) Change the **register descriptor** for R_x so it holds only x
 - b) Change the **address descriptor** for x so that its only location is R_x
 - c) Remove R_x from the **address descriptor** of any variable other than x
4. When processing a copy statement $x = y$:
 - a) If `LD  $R_y$ ,  $y$` is generated, manage descriptors using rule 1
 - b) Add x to the **register descriptor** for R_y
 - c) Change the **address descriptor** for x so that its only location is R_y

3. 对于实现三地址指令 $x = y + z$ 的 `ADD  $R_x$ ;  $R_y$ ;  $R_z$` 这样的运算而言
 - (a) 更改 R_x 的寄存器描述符，使其仅包含 x 。
 - (b) 更改 x 的地址描述符，使其只包含位置 R_x 。请注意， x 的地址描述符不包含 x 的内存位置。
 - (c) 从任何不同于 x 的变量的地址描述符中删除 R_x 。
 4. 当我们处理复制语句 $x = y$ 时，
 - (a) 如果有必要生成把 y 加载入 R_y 的加载指令，那么在生成加载指令并（按照规则
- 1) 像处理所有的加载指令那样处理完各个描述符之后，再进行下面的处理：
- (1) 将 x 添加到寄存器 R_y 的描述符。
 - (2) 更改 x 的地址描述符，使其只包含唯一的位置 R_y 。

Example

- a, b, c, d are live at the block exit
- t, u, v are temporaries, local to the block

$t = a - b$

$u = a - c$

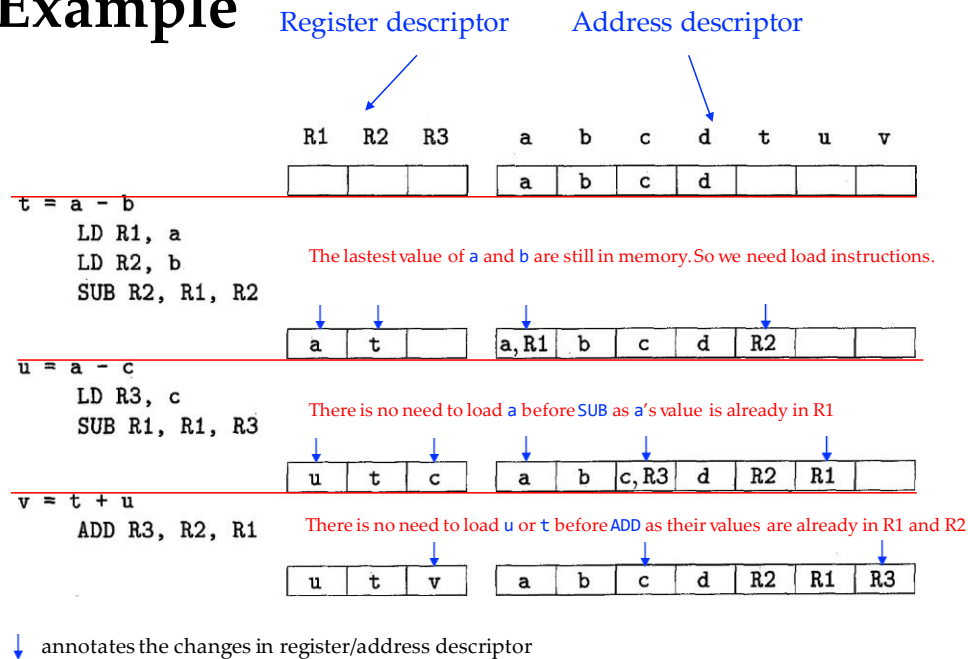
$v = t + u$

$a = d$

$d = v + u$

这里我们假设 t 、 u 和 v 是块的局部临时变量，而变量 a 、 b 、 c 和 d 是在块出口处活跃。由于我们还没有讨论 `getReg` 函数如何工作，我们将简单地假设有我们需要的寄存器，但是当不再需要寄存器的值时（例如，它只保存一个临时变量的值，且对这个临时变量的所有使用都已经处理完了），我们就重用这个寄存器。

Example



生成的所有机器代码指令的摘要如图 8.16 所示。该图还显示了每个三地址指令转换之前和之后的寄存器和地址描述符。

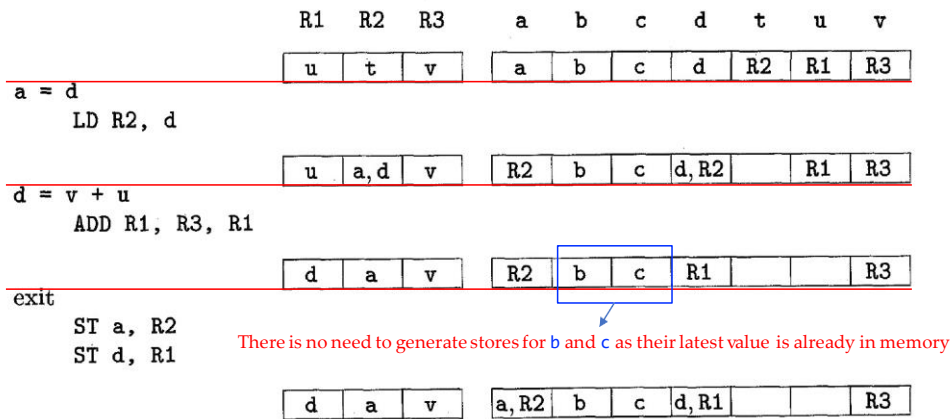
1. 因为寄存器中最初没有任何内容，我们需要为第一个三地址指令 $t = a - b$ ，生成三个指令。因此，我们看到 a 和 b 被加载存入寄存器 $R1$ 和 $R2$ ，并且 t 的值生成后存放在寄存器 $R2$ 中。请注意，我们可以将 $R2$ 用于 t 是因为块内不需要 $R2$ 中先前的值 b 。由于 b 可能在块退出时处于活跃状态，如果它不在自己的内存位置（如其地址描述符所示），我们就必须首先将 $R2$ 存储到 b 中。如果我们需要 $R2$ ，那么生成指令 $ST\ b\ R2$ 的决定将由 `getReg` 做出。

2. 第二条指令 $u = a - c$ 不需要加载 a 的指令，因为 a 已经在寄存器 $R1$ 中。我们可以重用 $R1$ 作为结果 u ，因为块内不再需要先前在该寄存器中的 a 值，并且如果块外需要 a ，则其值位于其自己的内存位置中。请注意，我们更改了 a 的地址描述符，以表明它不再位于 $R1$ 中，而是位于名为 a 的内存位置中。

3. 第三条指令 $v = t + u$ 仅需要一个加法指令。我们可以使用 $R3$ 来存放结果 v ，因为块内不再需要该寄存器中的 c 值，并且 c 在其自己的内存位置中有其值。

Example

Why do we generate store instruction only at block exit? Why only for a and d?



4. 复制指令 $a=d$ 需要一个指令来加载 d ，因为 d 不在寄存器中。我们展示了寄存器 $R2$ 的描述符，其中包含 a 和 d 。寄存器描述符中添加 a 是我们处理复制语句的结果，而不是任何机器指令的结果。

5. 第五条指令 $d = v + u$ 使用两个存放在寄存器中的值。由于 u 是一个临时变量，其值不再需要，因此我们选择重用其寄存器 $R1$ 来存放 d 的新值。请注意， d 现在仅位于 $R1$ 中，而不位于其自己的内存位置中。这同样适用于 a ，它在 $R2$ 中而不是在称为 a 的内存位置中。因此，我们需要为基本块的机器代码增加一个“尾声”，它把出口处活跃变量 a 和 d 的值保存到它们的内存位置。这就是图中的最后两条指令。

Design of the Function *getReg*

- **Goal:** Avoid too much data exchange with memory (by LD and ST)
- **Task:** Pick registers for operands and result of each three-addr inst
- **For the generic example $x = y + z$,** pick register R_y for operand y :
 - **Case 1:** If y is currently in a register, pick a register already containing y as R_y . Nothing else needs to be done.
 - **Case 2:** If y is not in a register, but there is a register that is currently empty, pick one such register as R_y
 - **Case 3:** What if y is not in any register and there is no register that is currently empty?

最后，让我们考虑如何实现三地址指令 I 的 $getReg(I)$ 。有很多选择，但也有一些绝对禁止的选择，这些选择会因丢失一个或多个活跃变量的值而导致错误的代码。我们从操作步骤的情况开始进行研究，对此我们再次使用 $x = y + z$ 作为通用示例。首先，我们必须为 y 选择一个寄存器，为 z 选择一个寄存器。这两次选择所面临的问题是相同的，因此我们将集中精力为 y 选择寄存器 R_y 。规则如下：

1. 如果 y 当前在寄存器中，则选择已经包含 y 的寄存器作为 R_y 。不需要生成一个机器指令来把 y 加载该寄存器。
2. 如果 y 不在寄存器中，但有一个寄存器当前为空，则选择一个这样的寄存器作为 R_y 。
3. 当 y 不在寄存器中并且没有当前为空的寄存器时，就会出现困难的情况。

Design of the Function *getReg*

y is not in register and there is no register that is currently empty:

- **Basic idea:** pick one allowable register and make it “safe” to reuse
- Let R be a candidate, and suppose v is a variable in R 's register descriptor
 - If v 's address descriptor says **we can find v somewhere besides R** , it is safe to reuse R
 - If **v is x** and **x is not the other operand z** , it is safe to reuse R *
 - If **v is not used later** (not live after the instruction), it is safe to reuse R
 - Otherwise, generate **ST v, R** to place a copy of v in its own memory location (*spill*, 溢出)
- If R holds multiple variables, repeat the process for each such variable v
 - At the end, R 's score is the number of generated store instructions. We can pick the candidate with the lowest score

* If v is x , since x is going to be redefined, it is ok to discard its old value. Why requiring x is not z ? Because as R is chosen to be reused, y 's value will be loaded into R . If x is z , that means the current value of z is in R , then it needs to be stored to avoid data loss.

当 y 不在寄存器中并且没有当前为空的寄存器时，无论如何，我们需要选择一个可行的寄存器，并且我们需要确保它可以安全地重用。

设 R 为候选者寄存器，并且假设 v 是 R 的寄存器描述符表明的位于 R 中的变量。我们需要确保 v 的值不是真正需要的，或者我们可以从其他地方获取 v 的值。可能的情况是：

- 如果 v 的地址描述符表明 v 位于 R 之外的某个位置，那么我们就完成了任务。
- 如果 v 是 x ，即由指令 I 计算的变量，并且 x 也不是指令 I 的其他运算分量之一（本例中为 z ），那么我们就完成了任务。原因是，在这种情况下，我们知道 x 的这个值永远不会再被使用，所以我们可以忽略它。
- 否则，如果稍后不使用 v （即，在指令 I 之后，不再使用 v ，并且如果 v 在块出口处活跃，那么 v 的值必然在基本块中被重新计算），那么我们就完成了任务。
- 如果前三种情况都不满足，则需要生成存储指令 **ST v** ； R 将 v 的副本放置在其自己的内存位置中。这种操作称为溢出。

由于 R 此时可能包含多个变量，因此我们对每个这样的变量 v 重复上述步骤。最后， R 的得分是我们需要生成的保存指令的个数。选择得分最低的寄存器（或者之一）。

Design of the Function *getReg*

- Picking register R_x for result x in $x = y + z$:
 - Since a new value of x is being computed, a register that holds only x is always an acceptable choice for x
 - If y is not used after the instruction, and R_y holds only y , then R_y can also be used as R_x (same for R_z)
- Pick registers R_x and R_y for $x = y$:
 - Pick R_y first (as above)
 - Then choose $R_x = R_y$

现在，考虑寄存器 R_x 的选择。其中的难点和可选项几乎与选择 R_y 时的一样，所以我们只提及其中的区别。

1. 由于正在计算 x 的新值，因此仅保存 x 的寄存器始终是 R_x 可接受的选择。即使 x 就是 y 或 z 之一，该语句仍然成立，因为我们的机器指令允许在一条指令中的两个寄存器相同。

2. 如果（像上面对变量 v 的描述那样） y 在指令 I 之后不再使用，且（在必要时加载 y 之后） R_y 仅保存了 y 的值，则 R_y 同时也可以用作 R_x 。对于 z 和 R_z 也有类似的选项。

最后要特别考虑的问题是当 I 是复制指令 $x = y$ 时的情况。我们用上面描述的方法选择寄存器 R_y ，然后是让 $R_x = R_y$ 。

Peephole Optimization (窥孔优化)

- A simple but effective technique for **locally improving target code** by examining a **sliding window** of target instructions (the *peephole*, 窥孔) and replacing the instruction sequences within the peephole with *shorter* or *faster* sequences*
 - Redundant-instruction elimination (冗余指令消除)
 - Flow-of-control optimizations (控制流优化)
 - Algebraic simplifications (代数简化)
 - Use of machine idioms (机器特有指令的使用)

* Can also be applied directly after intermediate code generation to improve the IR

虽然大多数生产编译器通过仔细的指令选择和寄存器分配来生成良好的代码，但也有一些编译器使用另一种策略：它们生成简单的代码，然后通过对目标代码应用“优化”转换来提高目标代码的质量。术语“优化”有些误导，因为不能保证生成的代码在任何数学测量下都是最优的。然而，许多简单的转换可以显着提高目标程序的运行时间或空间要求。

用于局部改进目标代码的一种简单但有效的技术是窥孔优化，它是检查目标指令的滑动窗口（称为窥孔）并且只要有可能就在窥孔内用更短或更快的指令替换窥孔内的指令序列。也可以在中间代码生成之后直接应用窥孔优化，以改进中间表示。

窥孔是程序上的一个小的滑动窗口。窥孔中的代码不需要是连续的，尽管某些实现确实需要这样做。窥孔优化的特点是每次改进又可能产生出新的优化机会。一般来说，为了获得最大利益，需要重复多次扫描目标代码。在本节中，我们将给出以下具有窥孔优化特征的程序转换示例：

冗余指令消除

控制流优化

代数简化

机器特有指令的使用。

Redundant Loads and Stores

- A naïve algorithm (not the one we introduced) may generate the following code:
 - `LD R0, a`
 - `ST a, R0`
- **We can delete the store instruction if there is no jump to it** (that is, the two instructions are in the same basic block)
 - If there is a jump to the store instruction, we could not be sure that the LD instruction is always executed before the store instruction (there might be other instructions putting a new value of `a` into `R0`)

如果我们在目标程序中看到指令序列 `LD R0, a`

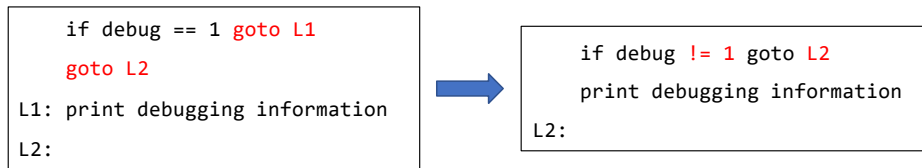
`ST a, R0`

我们可以删除保存指令，因为无论何时执行它，第一条指令都会确保 `a` 的值已经加载到寄存器 `R0` 中。

请注意，如果存储指令有标号，我们无法确定第一条指令总是在第二条指令之前执行，因此我们无法删除存储指令。换句话说，两条指令必须位于同一个基本块中，这样转换才能安全。这种性质的冗余加载和存储不会由上一节的简单代码生成算法生成。然而，像第 8.1.3 节中的原始的代码生成器可能会生成类似的冗余代码序列。

Unreachable Code

- **Example 1:** An unlabeled instruction immediately following an unconditional jump can be removed
- **Example 2:** Jumps over jumps (级联跳转)



Assuming there is no other jump to L1

How to optimize the code if we know debug is a constant 0?



另一个窥孔优化的机会是删除无法到达的指令。

紧随无条件跳转之后的不带标号的指令可以被删除。可以重复该操作以消除指令序列。例如，出于调试目的，大型程序中可能包含某些代码片段，仅当变量 `debug` 等于 1 时才执行这些代码片段。在中间表示中，此代码可能看起来像

```
if debug == 1 goto L1

goto L2

L1: print debugging information

L2:
```

一个明显的窥孔优化是消除级联跳转指令。因此，无论 `debug` 的值是什么，上面的代码序列都可以替换为

```
if debug != 1 goto L2

print debugging information

L2:
```

How to optimize the code if we know debug is a constant 0?

如果在程序开始时将 `debug` 设置为 0，则常量传播会将此序列转换为

```

if 0 != 1 goto L2

print debugging information

```

L2:

现在，第一条语句的计算结果始终为 `true`，因此该语句可以用 `goto L2` 替换。那么所有打印调试信息的语句都是不可达的，可以被逐一删除。

Flow-of-Control Optimizations

- Unnecessary jumps

<pre> goto L1 ... L1: goto L2 </pre>		<pre> goto L2 ... L1: goto L2 </pre>
--------------------------------------	---	--------------------------------------

<pre> if a < b goto L1 ... L1: goto L2 </pre>		<pre> if a < b goto L2 ... L1: goto L2 </pre>
--	---	--

Can we further eliminate L1: goto L2?



简单的中间代码生成算法经常产生目标为无条件跳转指令的无条件跳转指令，到达条件跳转的无条件跳转指令，或者达到无条件跳转指令的条件跳转指令。这些不必要的跳转指令可以通过以下类型的窥孔优化在中间代码或目标代码中消除。我们可以替换顺序

```

goto L1
...
L1: goto L2

```

替换为

```

goto L2
...
L1: goto L2

```

如果现在没有跳转到 L1 的指令，并且语句

L1: goto L2 之前是一个无条件跳转指令，则可以消除这个语句。

类似地，序列

```
    if a < b goto L1
```

```
    ...
```

```
L1: goto L2
```

可以替换为序列

```
    if a < b goto L2
```

```
    ...
```

```
L1: goto L2
```

Can we further eliminate L1: goto L2?

最后，假设只有一个达到 L1 的跳转指令，并且 L1 之前有一个无条件跳转指令，那么序列

```
    goto L1
```

```
    ...
```

```
L1: if a < b goto L2
```

```
L3:
```

可以被替换为序列

```
    if a < b goto L2
```

```
    goto L3
```

```
    ...
```

```
L3:
```

虽然两个序列中的指令数量相同，但在第二个序列我们有时可以跳过无条件跳转指令，但在第一个序列中的却不能。因此，第二个序列的执行时间优于第一个序列的执行时间。

Register Allocation and Assignment

- Instructions involving only register operands are faster than those involving memory operands
 - Efficient utilization of registers is vitally important in generating good code
- **Register allocation (寄存器分配)**
 - Decide at each point of a program what values should reside in registers
- **Register assignment (寄存器指派)**
 - Decide at each point of a program in which register each value should reside
- **There exist many strategies** for register allocation and assignment

仅涉及寄存器运算分量的指令比涉及存储器运算分量的指令更快。在现代机器上，处理器速度通常比内存速度快一个数量级或更高。因此，寄存器的有效利用对于生成良好的代码至关重要。本节介绍了各种策略，用于在程序中的每个点决定哪些值应驻留在寄存器中（寄存器分配）以及各个值应驻留在哪个寄存器中（寄存器指派）。

The Simple Strategy

- **Assign specific values to certain registers**
 - Assign array base addresses to a group of registers
 - Assign arithmetic computational values to another group
 - Assign the top of the stack to a fixed register ...
- **Advantage:** Simplifies the design and implementation of a compiler
- **Disadvantage:** Inefficient uses of registers
 - Certain registers may go unused, while many loads and stores are generated for the other registers
- It is reasonable to reserve a few registers for base addresses, stack pointers, and the like

寄存器分配和赋值的一种方法是将目标程序中的特定值分配给特定的寄存器。例如，我们可以决定将基地址分配给一组寄存器，将算术计算分配给另一组，栈顶指针分配给一个固定寄存器，等等。

这种方法的优点是简化了代码生成器的设计。但因为它的应用有太多限制，所以寄存器的使用效率低下；某些被占用的寄存器可能在大部分代码运行中未使用，同时却不得不生成很多不必要的而其他寄存器的加载和保存运算指令。然而，在大多数计算环境中还是要保留一些寄存器，这些被保留的寄存器可以被用作基址寄存器、堆栈指针寄存器或其他类似的用途，其他寄存器则由代码生成器在它认为适当的时候使用。

Global Register Allocation

- The earlier algorithm uses registers to hold values for the duration of a single basic block
 - All live variables were stored (if necessary) at the end of each block
- To save stores and loads, we can **assign registers to frequently used variables** and keep these registers consistent across block boundaries (**globally**)
 - E.g., assign certain registers to hold the most active values in a loop
- Another strategy is to estimate the benefits of putting a variable in register (via static analysis or profiling) and allocate registers according to the estimations

第 8.6 节中的代码生成算法在单个基本块的运行期间使用寄存器来存放值。然而，在每个基本块的结尾处，所有活跃变量的值都被保存到内存中。为了省略一部分这样的保存及相应的存储指令，我们可以把一些寄存器指派给频繁使用的变量，并使得这些寄存器在不同基本块中的（即全局的）指派保持一致。

由于程序大部分时间都花在它的内部循环上，因此一个自然的全局寄存器指派方法是尝试在整个循环中将频繁使用的值保留在固定寄存器中。暂时假设我们知道一个流图的循环结构，并且我们知道在基本块中计算的哪些值会在该块之外使用。下一章将介绍计算这些信息的技术。

全局寄存器分配的一种策略是分配一些固定数量的寄存器来保存每个内部循环中最活跃的值。不同循环中选择的值可能不同。尚未分配的寄存器可用于保存一个块的局部值，如第 8.6 节所示。这种方法的缺点是固定数量的寄存器并不总是恰好等于用于全局寄存器分配的最佳数量。然而，该方法实现起来很简单，并在 Fortran H 中使用，Fortran H 是 IBM 在 20 世纪 60 年代末为 360 系列机器开发的优化 Fortran 编译器。

使用早期的 C 编译器，程序员可以通过使用寄存器声明显式地进行一些寄存器分配，以便在过程运行期间将某些值保留在寄存器中。明智地使用寄存器声明确实加快了许

多程序的速度，但应鼓励程序员在分配寄存器分配之前首先获取程序运行时刻特征并确定程序的热点代码。